

Interação com APIs

08



Segurança na Integração

Protegendo a comunicação entre Front-end e APIs

Contextualização Profissional

Em ambientes reais, sistemas lidam com dados sensíveis de usuários, transações financeiras e informações estratégicas. Entender segurança não é opcional — é um requisito profissional.

 Internet Banking

Transações e saldos protegidos

 Prontuário Médico

Dados clínicos confidenciais

 ERP Corporativo

Operações e finanças internas

 Sistema Escolar

Notas, dados de alunos

 Marketplace

Pedidos, pagamentos, endereços

 Portal Administrativo

Acesso restrito a gestores

Nem toda informação pode ser pública. Sistemas modernos exigem controle de acesso rigoroso em cada endpoint.

Situação de Aprendizagem

A Pergunta-Problema

O Cenário

Uma empresa desenvolveu um sistema corporativo com área de login, painel administrativo, consulta de dados restritos e operações protegidas. O front-end Angular precisa autenticar o usuário, armazenar token, enviar credenciais nos requests, restringir rotas e tratar sessões expiradas.

 **Pergunta-Problema:** Como garantir que somente usuários autorizados acessem recursos protegidos de uma API?

Esta é a questão central que conduzirá toda a aula. Ao final, você será capaz de responder com código funcional e boas práticas consolidadas.

Confidencialidade

Somente pessoas autorizadas podem acessar determinados dados. É o princípio que impede que informações sensíveis sejam expostas a quem não tem permissão.

Exemplos de Dados Confidenciais

 Senha	 CPF
 Salário	 Dados Bancários
 Info. Estratégicas	

Controles Aplicados

- Login
Identifica quem está acessando o sistema
- Criptografia
Protege dados em trânsito e em repouso
- Autorização
Garante acesso apenas ao necessário

Integridade

Os dados não podem ser alterados de forma indevida. Qualquer modificação não autorizada compromete a confiabilidade do sistema — e pode gerar prejuízos graves em contextos financeiros ou legais.

Dados que Exigem Integridade

- **Valor de Pix** — nenhum centavo pode ser alterado
- **Nota Fiscal** — documento fiscal com validade jurídica
- **Pedido de compra** — itens, quantidade e preço
- **Contrato digital** — cláusulas não podem ser adulteradas
- **Prontuário médico** — registros de saúde do paciente

Mecanismos de Integridade

01

Assinatura Digital

Garante autoria e não repúdio

02

Hash (SHA-256)

Detecta qualquer alteração no dado

03

Validação no Back-end

Regras de negócio que rejeitam dados inválidos

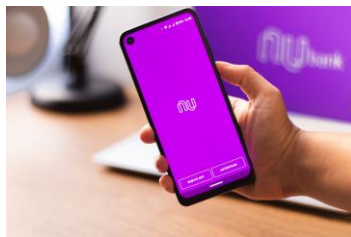
04

Controle Transacional

Operações atômicas no banco de dados

Disponibilidade

O serviço precisa estar acessível quando o usuário precisar. Sistemas críticos não podem ter interrupções — cada minuto de indisponibilidade representa perda financeira ou risco à vida.



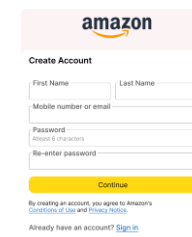
App Bancário

Transações financeiras exigem disponibilidade 24/7



Sistema Hospitalar

Prontuários e monitoramento de pacientes não podem falhar



E-commerce

Quedas em datas comerciais geram perdas milionárias



APIs Críticas

Falhas em APIs impactam todos os sistemas integrados

❌ **Riscos à Disponibilidade:** falhas de infraestrutura, indisponibilidade programada mal planejada, sobrecarga de requisições e ataques DDoS.

Autenticação × Autorização

AUTENTICAÇÃO VS. AUTORIZAÇÃO

Autenticação

Quem é você?

É o processo de verificar a identidade do usuário. O sistema valida as credenciais (usuário e senha, biometria, token) e confirma que aquela pessoa existe e é quem afirma ser.

 Resultado: o usuário recebe um token de sessão.

Autorização

O que você pode fazer?

É o processo de verificar as permissões do usuário autenticado. Mesmo após o login, cada ação é verificada — nem todo usuário autenticado tem acesso a todas as funcionalidades.

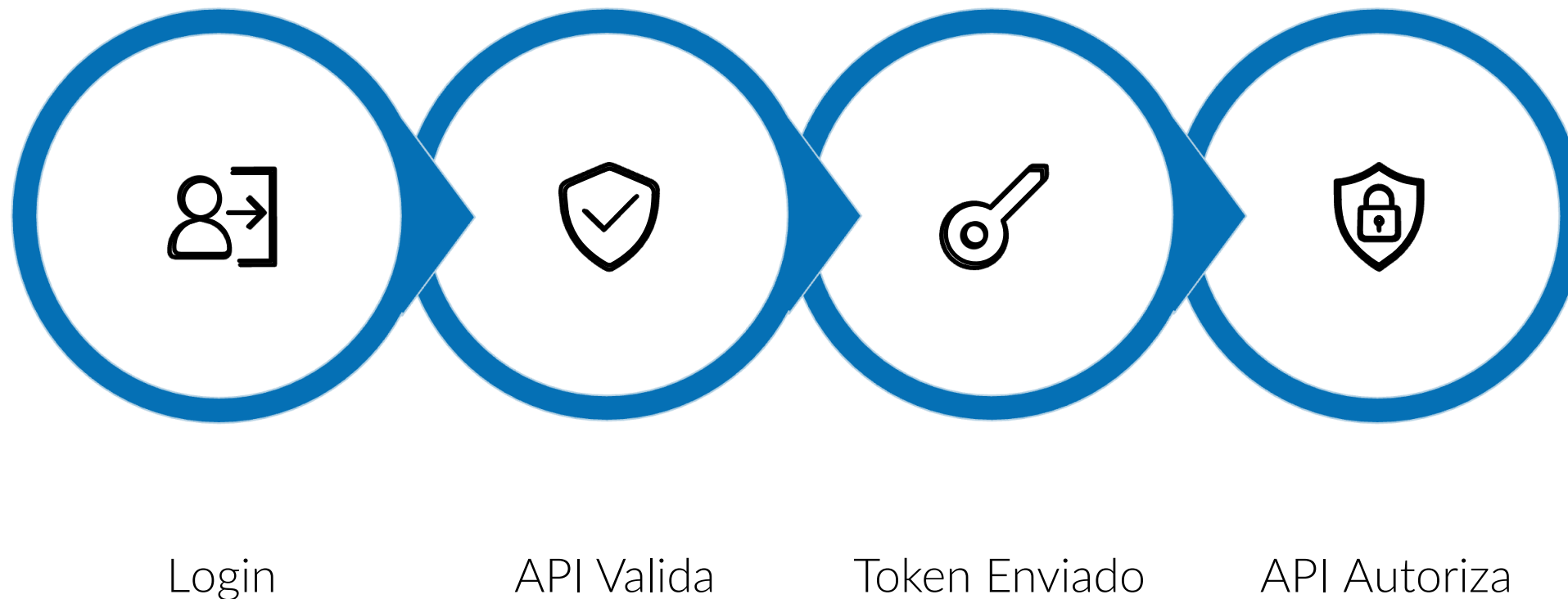
 Usuário autenticado ≠ Administrador. O papel (role) define o que cada um pode acessar.

Autorização
Verifica permissões e libera
ou bloqueia recurso.



Tokens: Credencial Digital de Sessão

Um **token** é uma string gerada pela API após validação das credenciais. Ele substitui a sessão tradicional baseada em cookies em aplicações modernas com APIs REST.



Header de Autenticação

Em cada requisição autenticada, o token é enviado no cabeçalho H

```
Authorization: Bearer token_aqui
```

Por que Bearer? O prefixo "Bearer" indica que quem possui o token tem autorização. A API valida o token e libera ou bloqueia o acesso ao recurso solicitado sem precisar consultar banco de dados de sessão.

Prática com DummyJSON

Login com API Real

Utilizaremos a API pública **DummyJSON** para simular autenticação. Ela retorna um JWT real que pode ser usado nos exercícios práticos da aula.

Endpoint de Login

```
https://dummyjson.com/auth/login
```

Payload (corpo da requisição)

```
{
  "username": "emilys",
  "password": "emilypass"
}
```

Resposta da API

```
{
  "accessToken": "jwt...",
  "refreshToken": "jwt...",
  "username": "emilys"
}
```

Com este JWT em mãos, o front-end Angular armazena a credencial e a envia em cada requisição subsequente às rotas protegidas da API.

✅ **Fluxo:** Login → JWT → Sessão ativa

AuthService — Realizando o Login

No Angular, o serviço de autenticação centraliza a lógica de login. O método `login()` realiza um `POST` para a API e retorna o Observable com a resposta — incluindo o `accessToken` e o `refreshToken`.

Código do AuthService

```
login(credentials: Login) {  
  return this.http.post(  
    'https://dummyjson.com/auth/login',  
    credentials  
  );  
}
```

O que este código faz?

01

Recebe as credenciais

Objeto do tipo `Login` com username e senha

02

Executa POST

Envia para o endpoint de login da API

03

Retorna Observable

O componente se inscreve e recebe o token

❏ O `HttpClient` do Angular deve estar importado no módulo ou via `provideHttpClient()` no `app.config.ts` para que o serviço funcione corretamente.

Próxima camada de segurança: HTTP Interceptor

O **Interceptor** é um mecanismo do Angular que intercepta automaticamente todas as requisições HTTP e injeta o token de autenticação no cabeçalho. Isso centraliza a autenticação e evita repetição de código nos serviços.

Header Injetado pelo Interceptor

```
Authorization: Bearer ${accessToken}
```

Como funciona

- Intercepta o Request
Todo HTTP request passa pelo interceptor
- Lê o Token
Recupera do `localStorage` ou serviço de sessão
- Clona e Envia
Adiciona o header e repassa o request modificado

Vantagem do Interceptor

Sem o interceptor, cada chamada HTTP precisaria manualmente adicionar o header de autenticação, gerando código duplicado e maior risco de falha. Com o interceptor, a autenticação é **centralizada e transparente**.

- ✓ O front-end centraliza autenticação em um único ponto.

DummyJSON retorna JWT real.
No projeto atual, o token já é armazenado em `sessionStorage`. O próximo passo arquitetural é automatizar o envio do header via interceptor.

Próxima camada: Route Guard

O **Route Guard** impede que usuários não autenticados acessem rotas protegidas do sistema. Quando o guard detecta ausência de token válido, redireciona automaticamente para a tela de login. Como já temos `isAuthenticated()`, o próximo passo é usar esse método em um Guard para proteger rotas.

Rotas que Devem ser Protegidas

 Dashboard

Painel principal

 Admin

Área administrativa

 Perfil

Dados do usuário

 Configurações

Preferências do sistema

Por que proteger rotas no Angular?

Mesmo que a API bloqueie requisições sem token, a interface também precisa controlar o acesso. Um usuário não autenticado não deve nem mesmo ver o painel administrativo — isso faz parte de uma boa experiência segura.



Atenção: o Guard de rota não substitui a validação da API. As duas camadas são complementares e necessárias.

Boas Práticas de Segurança

✗ Nunca Faça

Token na URL (query string)

Senha em `localStorage`
em texto puro

Usar HTTP sem
criptografia (não HTTPS)

Expor `secrets` no código-
fonte

Logar credenciais no
console

✓ Sempre Faça

HTTPS em todos os
ambientes

Definir expiração do token

Implementar logout que
limpa sessão

Validar dados no back-end

Aplicar princípio do
menor privilégio

Mini Projeto: Sistema de Login Angular

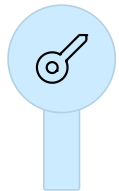
Desenvolva uma aplicação Angular completa com autenticação integrada à API ReqRes. O projeto deve contemplar todo o fluxo de segurança estudado na aula.



Autenticar com a API

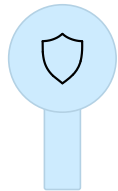
Tela de login com POST para <https://dummyjson.com/auth/login>

username: emilys password: emilyspass



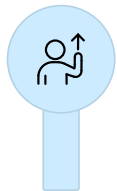
Receber e Salvar Token

Armazenar o token retornado de forma segura na sessão



Verificar sessão

Permitir navegação para sessão iniciada



Logout

Limpar token e redirecionar para login ao sair

Reflexão Técnica

Debate em Sala

Antes de encerrar, vamos aprofundar o raciocínio com questões que aparecem em situações reais de desenvolvimento. Discuta com o grupo:

Expiração do Token

O que acontece quando o token expira? Como o front-end deve tratar um retorno `401 Unauthorized` da API?

Onde Armazenar?

`localStorage`, `sessionStorage` ou memória? Quais são os riscos de cada abordagem em termos de segurança?

Interceptar 401

Como o interceptor pode tratar respostas 401 de forma centralizada — renovando token ou redirecionando para login?

Login vs. Autorização

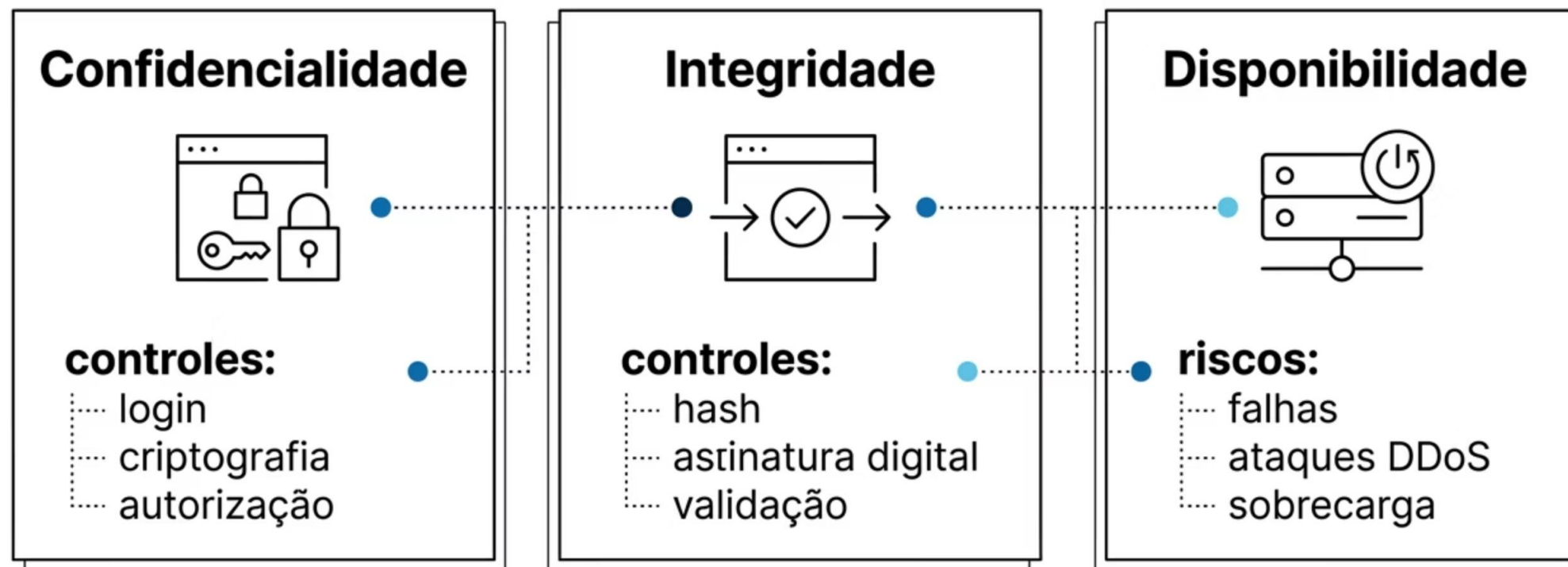
Qual a diferença prática entre estar logado e ter permissão de acesso a um recurso específico?

Front-end é Suficiente?

O Route Guard garante segurança? Por que a validação da API é indispensável mesmo com proteção no front?

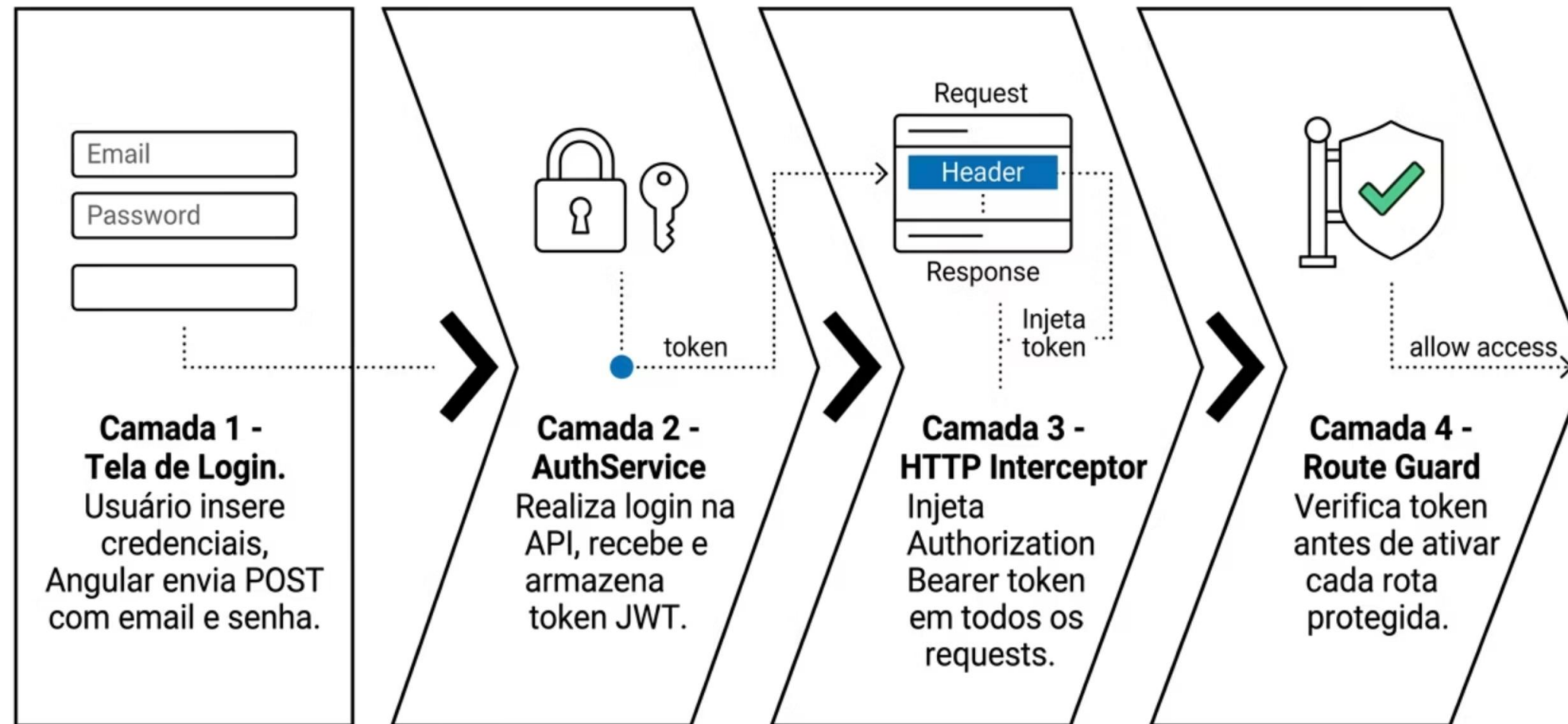
CIA Triad em Revisão Visual

Três pilares da segurança da informação



Os três pilares da CIA Triad formam a base de qualquer estratégia de segurança em sistemas que consomem APIs. Em Angular, implementamos controles para cada um deles: autenticação garante confidencialidade, validação protege a integridade, e boas práticas de infraestrutura sustentam a disponibilidade.

Arquitetura de Segurança no Angular



Cada camada adiciona um nível de proteção. O Route Guard protege a navegação, o Interceptor protege as requisições, o AuthService gerencia a sessão, e a API valida o token no servidor. Segurança é sempre multicamadas.

Evidência de Competência

Ao concluir esta aula, você demonstrou ter atingido as seguintes competências previstas pela MSEP:



Explica a CIA Triad

Confidencialidade, Integridade e Disponibilidade



Diferencia Auth × Authz

Autenticação (identidade) vs. Autorização (permissão)



Compreende Tokens

Fluxo login → token → sessão em APIs REST



Envia Auth Header

Authorization: Bearer via Interceptor Angular



Protege Rotas Angular

Route Guard bloqueando acesso não autorizado



Aplica Boas Práticas

HTTPS, expiração, menor privilégio, logout seguro

Para Aprofundar o Conhecimento

Documentação Oficial

- Angular HTTP Client
Documentação oficial do `HttpClient` e Interceptors em *angular.io*
- Angular Route Guards
Guia completo de `CanActivate` e proteção de rotas
- OWASP Top 10
Principais vulnerabilidades em aplicações web — referência global de segurança

API de Prática

 DummyJSON

API pública para testes de login e autenticação com JWT real

 JWT.io

Ferramenta para visualizar, decodificar e validar tokens JWT no navegador

 O uso de APIs públicas como DummyJSON permite praticar autenticação em ambiente seguro, sem exposição de dados reais.

